House Sale Prices Prediction

Cristina Papi, Marco Rosato, Rowyda Askalani
Universita‘ degli Studi di Milano Bicocca, CdLM Data Science

Abstract

This project presents a comprehensive benchmark of machine learning models using Knime to predict house sale prices in King County, using a dataset of 21,613 sales from 2014–2015. The methodology involved a data preparation pipeline, including feature engineering, logarithmic transformations, and target encoding of zip codes. Two feature selection processes were employed to identify the most significant variables for both linear and non-linear models. Five models were developed and evaluated: Linear Regression, Polynomial Regression, Random Forest, Gradient Boosted Trees, and a Multilayer Perceptron, with non-linear models fine-tuned via Bayesian hyperparameter optimization.

Results demonstrate that the Gradient Boosting model achieved the best results, with a Mean Absolute Error (MAE) of $66,549 on the test data. The study also confirms that, thanks to alternative pipelines, every step that was developed for our full model was critical for predictive performance. Additionally, we have noticed that the Random Forest provides a strong alternative where prediction stability is crucial.

Keywords: House Price Prediction, Machine Learning, Regression, Gradient Boosting, Random Forest, Feature Engineering, Hyperparameter Optimization.

Contents

1 Introduction

Predicting house prices is a well-known problem in the field of data science and machine learning, with significant practical implications in the real world. Accurate prediction of house prices is crucial for real estate agents, buyers, investors, and urban planners. However, pricing is influenced by many heterogeneous variables (such as location, square footage, condition, etc.), which makes the problem complex.

This project aims to benchmark different models used to predict house sale prices. By analyzing real-world housing data, the models learn patterns and relationships between various property features, allowing them to estimate how much a house would cost based on the input information.

To carry out this project, we used the software "Knime" [3], which allowed us to design and implement the entire data processing and modeling workflow.

2 Chosen Dataset

The dataset contains house sale prices for King County, which includes Seattle. It includes homes sold between May 2014 and May 2015. It is obtained from the Kaggle site [1]. In the dataset, there are 21613 rows and 21 columns that describe the main features of houses. The variables are:

`id`, `date`, `price`, `bedrooms`, `bathrooms`, `sqft_living`, `sqft_lot`, `floors`, `waterfront`, `view`,`condition`,`grade`, `sqft_above`, `sqft_basement` `yr_built`, `yr_renovated`, `zipcode`, `lat`, `long`, `sqft_living15`, `sqft_lot15`.

The description of each variable can be found on this website [2].

The dependent variable is `price`, in fact the main purpose is to predict house prices. In the following Figure 1 we can see the dependent variable distribution:

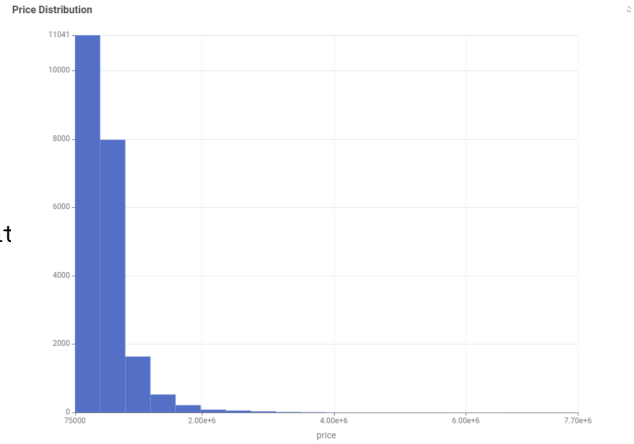


Figure 1: Distribution of the dependent variable price

3 Data Preparation

3.1 Feature Engineering

We created new features in an effort to obtain meaningful features that can be used within the project for training. The features that we have created are the following:

- `Age`: based on whether or not the house got renovated we assign it an age.
- `ratio_living_size15`: This measures how the house's living space compares to its neighborhood average.
- `ratio_lot_size15`: This measures how the house's lot size compares to its neighborhood average.
- `living_lot_ratio`: This indicates how much land the house occupies
- `Volume`: This variable acts as some kind of volume to indicate the house's scale.

after the new features have been created, we dropped the column `yr_renovated`, `yr_build` (since they were used in the previous step) along side the column `id` and `date`. We decided to exclude the column `date` since the dataset covers a time span of only one year, which limits the possibility of capturing trends, seasonality, or time-dependent patterns. We observed that the dataset had no missing values. However, we identified and removed a small number of anomalous rows where the sale price did not correspond logically to the number of bedrooms and bathrooms. Additionally, we also adjust an instance

where the number of bedrooms were incorrectly stated as 33 instead of 3. Where the threshold of the skewness for the transformation is more than 1, we applied a logarithmic transformation to all continuos variables to reduce skewness and mitigate the influence of outliers. This transformation also helps normalize the distribution of values, which can improve the performance and stability of the machine learning models during training.

3.2 Train/Test Split

The data is split into training and testing where the training data takes up `70%` and the test data is assigned only `30%` by random. The splitting of the dataset into train and test is useful as we will need the model to train on as much data as possible in order to make the correct predictions on the test data.

3.3 Target Encoding

To make better use of the variables available, we decided to perform target encoding on the variable `Zipcode`. To better understand how the prices are affected based on the area in which they can be found, on the training data, we grouped, each zip code and perform the median of the price of each of them. The median was a conscious choice in this case, this is because the price variable included some skewed values which would have greatly affected the mean. Subsequently, as before, we logarithmically transformed the new target encoding variable to reduce its skewness.

We plotted the target encoding distribution to understand the median price of the different zones of the city, to comprehend which is the most expensive area. As we can see from the Figure 2, which was produced in R Studio[5] using the Leaflet package[4], the closer we get to the centre of Seattle the higher the median price becomes.

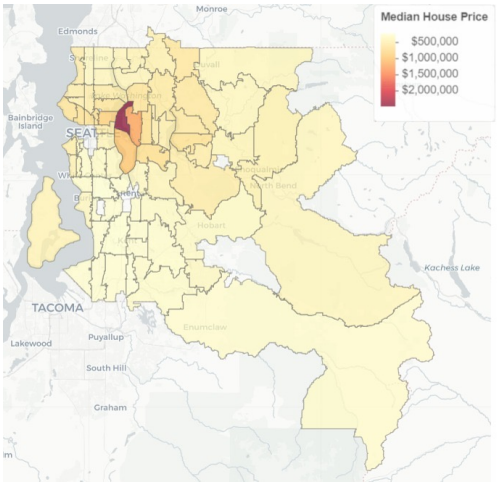


Figure 2: Target encoding variable distribution

4 Feature Selection

4.1 Feature Selection Process

Before training the final models on the training data and evaluating them on the test set, two feature selections were performed to identify the most significant covariates, one for non-linear models and the other for linear regression. The forward feature selection procedure was used, selecting the features that contributed to minimizing the MAE (Mean Absolute Error).

A Random Forest (for the non linear models) and a Linear Regression (for the linear model) were trained and evaluated using the k-fold cross-validation technique (k equal to 5). Prior to training, all variables were normalized. To prevent data leakage, this normalization was applied separately within each fold of the cross-validation, using only the training data of that fold. This normalization procedure was used **across all** the cross-validation processes of the project. For each subset of features, performance metrics were computed, and the average cross-validation performance was used as the criterion for ranking feature importance.

After model training, the exponential of the predicted log-transformed price was calculated in order to recover the actual price values and make the results interpretable.

Finally, we manually selected the best-performing feature set for both linear and non-linear models, finding a balance between the MAE and the num-

ber of features retained.

4.2 Feature Selection for non linear models

The chosen best model was the one with 11 variables (`waterfront`, `view`, `condition`, `grade`, `lat`, `long`, `age`, `log_sqft_living`, `log_sqft_lot`, `log_zip_median_price`), The MAE of $66595. This means that, on average, the model estimates the house price with an error of approximately $66595.

4.3 Feature Selection for linear models

This time, the chosen best model was the one with 13 covariates and a MAE equal to $81860. The features are: `bedrooms`, `bathrooms`, `waterfront`, `view`, `condition`, `grade`, `lat`, `long`, `age`, `log_sqft_living`, `log_sqft_above`, `log_sqft_basement`, `log_zip_median_price`.

5 Training-testing of the models

5.1 Models

Machine learning algorithms aim to learn from available data different types of information that allow making predictions or decisions in certain fields. The analysis used five learning algorithms, described as follows:

- **Linear Regression**: It predicts a continuous target variable based on one or more input features. It fits a straight line to minimize the sum of squared differences between actual and predicted values;
- **Polynomial Regression**: It extends linear regression by adding polynomial terms to capture non-linear relationships;
- **Random Forest**: A versatile ensemble method for classification and regression. It builds an ensemble of decision trees. It averages or votes over their outputs to reduce overfitting and improve accuracy;
- **Gradient Boosted Trees**: An ensemble method for classification and regression. It builds trees sequentially, where each new tree corrects the errors of the previous ones. Trees are combined to minimize a loss function using gradient descent;
- **MultiLayer Perceptron - MLP**: A feedforward neural network for classification, regression, and pattern recognition. The algorithm that was used for the MLP in this case is the Resilient Backpropagation (Rprop).

5.2 Cross-Validation

In order to check how the models are performing on unseen validation data, we rely on Cross Validation. The implementation process for all models is equivalent: we splitted the data into five folds, setted it to random sampling, iterated over the folds, and printed the results of the performance of every fold. This procedure was used in the feature selection process as well as the hyper-parameter optimization.

5.3 Hyper-parameter Optimization

In order to tune the non-linear models, we perform Hyper-parameter optimization. The idea is to allow the models to iterate over the parameters space in order to find the best-performing configuration based on the training data.

The procedure is consistent across all non-linear models. We begin by defining the hyper-parameters to be optimized, specifying a search range for each. The initial search strategy is set to Random Search, which is repeated three times using progressively narrower parameter ranges to refine the results. Following this, we apply Bayesian Optimization to further explore the hyper-parameter space and identify the optimal configuration.

For each of the models the following parameters and their results were defined:

- **Polynomial Regression**: Degree(6)
- **Random Forest**: `nrModels` (1136), `maxLevels`(17), `minNodeSize` (20), `minChildSize` (2), `dataFraction` (0.997), `columnFractionPerTree` (0.696)
- **Gradient Boosted Trees**: `nrModels` (490), `learningRate` (0.105), `maxLevels` (5), `minNodeSize` (95), `minChildSize` (37), `dataFraction` (0.976), columnFractionPerTree (0.876)
- **MultiLayer Perceptron - MLP**: Max num iteration (1181), Num hidden layer (3), Num hidden neuron (37)

5.4 Final Models' training and testing

Once the hyper-parameters have been tuned, the final model is trained on the entire training dataset. After training, the model proceeds to the testing phase, where it generates price predictions for the test set.

In order to view the price correctly in dollars, we apply the exponential function to the original price and the predicted prices. This assists in viewing the metrics in dollars in order to evaluate the model accordingly.

5.5 Alternatives

To ensure the effectiveness of our workflow, we tested three simplified variants of the Linear Regression model: one without the logarithmic transformation; one without both feature engineering and the logarithmic transformation; and one without feature engineering, logarithmic transformation, and feature selection.

These variants progressively remove key preprocessing steps to assess their individual impact on model performance and determine whether the full workflow offers a measurable advantage.

6 Results

6.1 Test Results

Table 1 presents a benchmark of the test results for the five models evaluated in our analysis, along with their respective performance metrics. We specifically highlight the best-performing values for the Mean Absolute Error (MAE), which was the metric chosen for minimization. MAE was selected due to its interpretability, its use of the same units as the target variable, and its reduced sensitivity to outliers compared to other metrics such as Mean Squared Error (MSE). It provides a clear indication of the average prediction error in dollar terms, aligning well with the objectives of this project.

The best value of the MAE is equal to $66549.129, which corresponds to the Gradient Boosting model. This indicates that, on average, its predictions deviate from the actual values by approximately 66.5k units, suggesting superior performance compared to the others models evaluated. As for the Random Forest and MLP models, their MAE values are 71,739.53 and 71,295.54, respectively. These results are relatively close to each other and only slightly higher than that of Gradient Boosting, suggesting comparable but slightly lower performance.

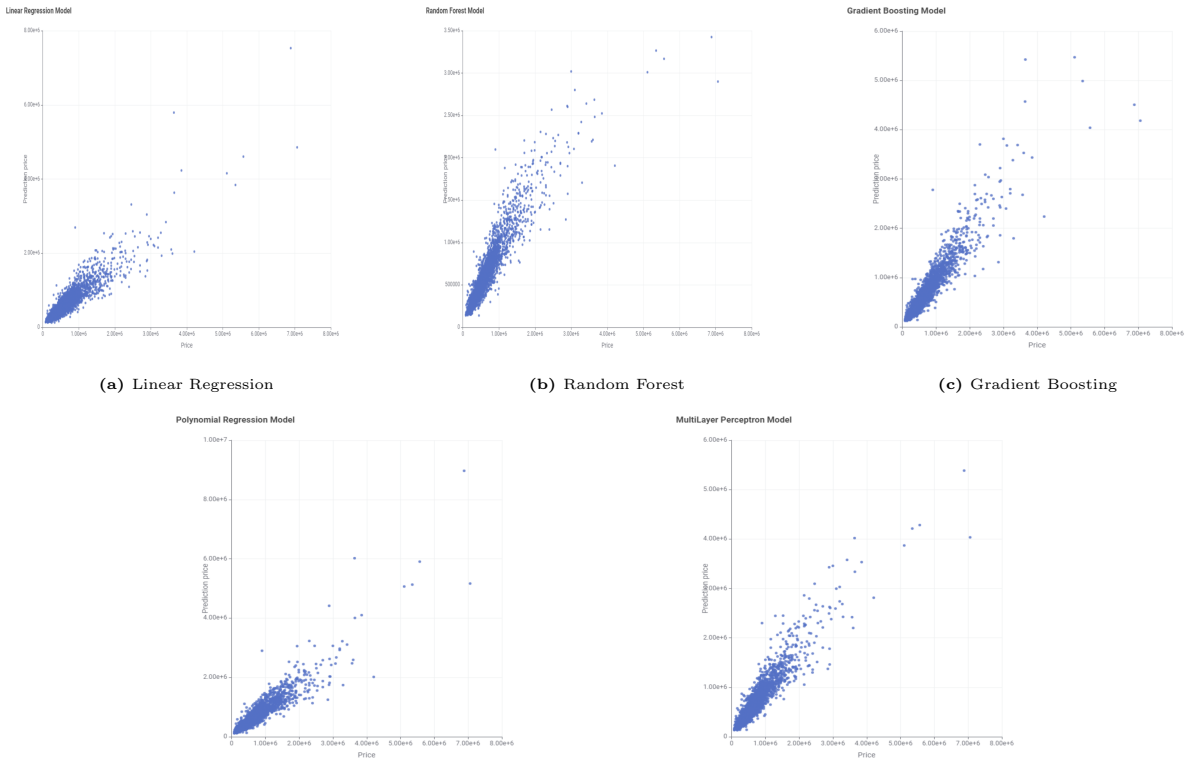
The worst-performing models were Linear Regression and Polynomial Regression, which exhibited higher prediction errors.

RowID	Linear Regression	Random Forest	Gradient Boost	MLP	Polynomial Regression
R^2	0.843	0.848	0.892	0.885	0.872
MAE	88,302.344	71,739.531	**66,549.129**	71,295.535	76,156.36
MSE	24,513,205,277.254	23,769,615,995.672	16,871,217,831.658	17,947,623,257.873	20,051,748,961.001
RMSE	156,566.935	154,173.98	129,889.252	133,968.74	141,604.198
MSD	-11,449.184	-14,679.691	-1,221.809	-7,993.955	-8,183.109
MAPE	0.155	0.125	0.12	0.128	0.136
Adjusted R^2	0.843	0.847	0.892	0.885	0.871

Table 1: Model Comparison Metrics

6.2 Price VS Predicted Price

In order to understand the relation between the actual price and the predicted one, we displayed the performance of each model through a scatter plot. As opposed to the results of the metrics that indicate that the gradient Boosting is the best performing, we can observe from Figure 3 that the visually best performing model is the Random Forest model. This means that while Gradient Boosting might be slightly better on average, it is also prone to making a few wildly inaccurate predictions for high-priced items as observed from the figures. While the Random Forest model is more conservative, robust, and it's less likely to produce these extreme outliers.



(a) Linear Regression **(b)** Random Forest **(c)** Gradient Boosting

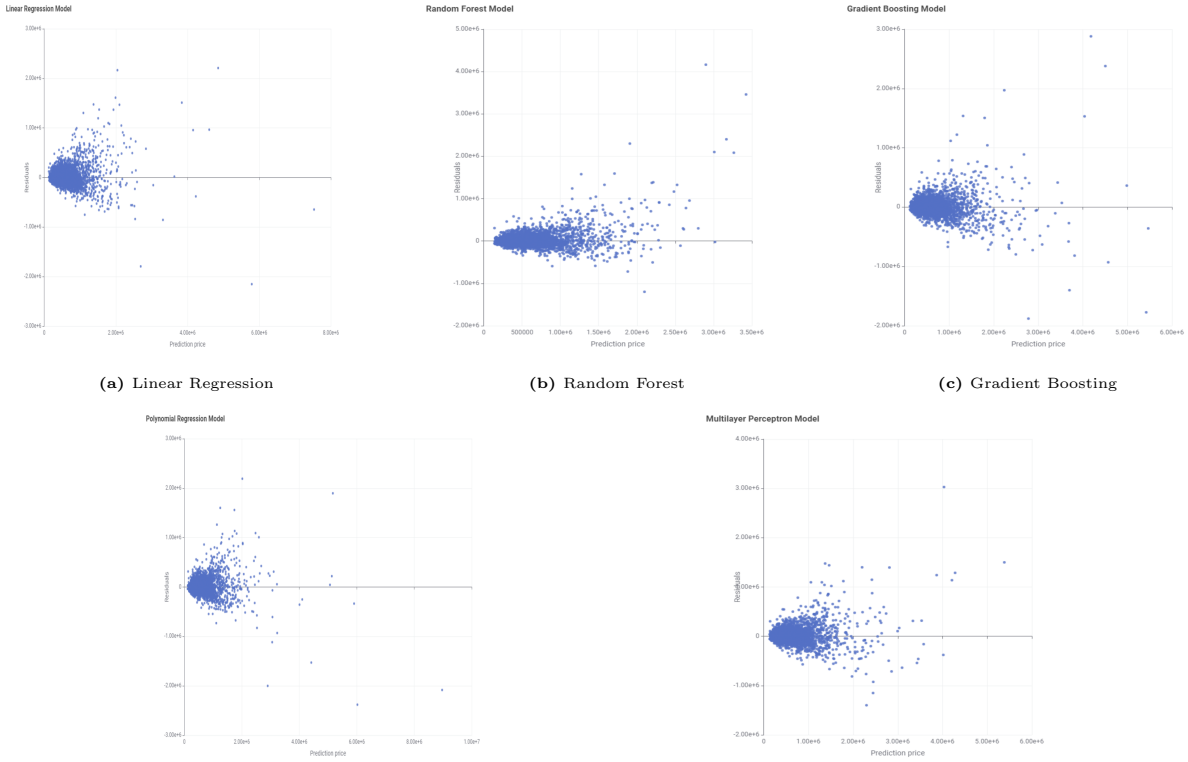
(d) Polynomial Regression **(e)** Multilayer Perceptron

Figure 3: Scatter plots: True Price (X) vs. Predicted Price (Y) for various models

6.3 Residuals

In order to understand whether the models under or over predict the different parts of it's output range, we decided to visualize the residuals vs the predicted price of each model by relying on scatter plots as shown in Figure 4. From what we can observe, all the scatter plots suffer from heteroscedasticity. For low prediction prices, the residuals are small and tightly packed around the zero line. This means the models are quite accurate for lower-priced items. As the prediction price increases, the spread of the residuals becomes much larger. This means the model's

errors are significantly larger for higher-priced items. However, despite that fact the Gradient Boosting Model is the best performing one as it has the points appear slightly tighter and more concentrated around the zero line than the other models.



(a) Linear Regression **(b)** Random Forest **(c)** Gradient Boosting

(d) Polynomial Regression **(e)** MLP Model

Figure 4: Scatter plots: Predicted Price (X) vs. Residuals(Y) for various models

6.4 Cross-Validation Results

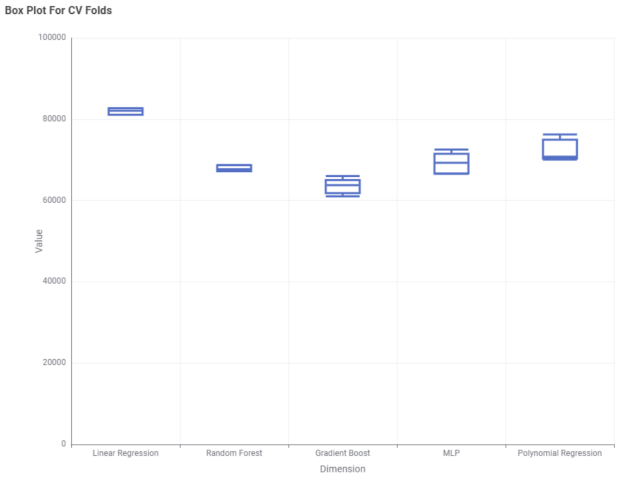


Figure 5: A box plot figure showcasing the performance of all the models on the CV folds.

To better visualize the performance of each fold for the different models, we plotted their performance through box plot, showcasing which models performed best on the folds, which can be seen in figure 5. In the figure, we can observe that the best performing model is indeed the Gradient Boosting as it is the model with lowest MAE on all folds. Additionally, we can confirm that the Random Forest model has the smallest variation, emphasizing the residuals result.

6.5 Alternatives Results

To validate the effectiveness of our workflow, we compared it against several alternative approaches by analysing their performance metrics, specifically focusing and minimizing the Mean Absolute Error (MAE). For simplicity, we assigned abbreviated names to each model:

- **LR**: Complete pipeline linear regression
- **LMWLT**: Linear Model without Logarithmic Transformation

- **LMWFELT**: Linear Model without Feature Engineering and Logarithmic Transformation
- **LMWFELTFS**: Linear Model without Feature Engineering, Logarithmic Transformation, and Feature Selection

As shown in Table 2, the full model (LR) outperformed all alternatives, achieving the lowest MAE of 88,302.34 dollars. This result confirms that the complete pipeline helps in achieving a more accurate predictions among the models tested.

RowID	**LR**	**LMWLT**	**LMWFELT**	**LMWFELTFS**
R^2	0.843	0.783	0.682	0.686
MAE	88,302.344	104,092.22	125,926.383	126,767.018
MSE	24,513,205,277.254	33,897,330,384.47	43,169,256,390.297	42,648,131,946.071
RMSE	156,566.935	184,112.277	207,772.126	206,514.242
MSD	-11,449.184	140.995	-3,450.867	458.514
MAPE	0.155	0.196	0.249	0.255
Adjusted R^2	0.843	0.782	0.682	0.686

Table 2: Comparison of the different alternatives

7 Conclusions

This project addressed the prediction of house prices in King County through a machine learning pipeline applied to a real-world dataset. After thorough preprocessing and feature selection, several models were trained and optimized using cross-validation and hyperparameter tuning.

The results showed that Gradient Boosting achieved the best overall performance, with the lowest Mean Absolute Error (MAE) of approximately $66,549, indicating strong predictive accuracy. However, our benchmark analysis highlighted a key trade-off between accuracy and robustness: while Gradient Boosting performed best on average, Random Forest proved more reliable in avoiding large errors on high-value properties, making it a strong candidate for risk-sensitive applications.

These findings support the feasibility of using machine learning for real estate price estimation and lay the groundwork for future developments, such as integrating additional data sources, refining spatial analysis, or deploying the models in practical valuation tools. As future work, exploring heterogeneous ensemble methods, which combine different model types, could further enhance predictive performance and better balance generalization and robustness across diverse property segments.

References

[1] House prediction sales. https://www.kaggle.com/datasets/harlfoxem/housesalesprediction.

[2] House prediction sales variables explaination. https://rstudio-pubs-static.s3.amazonaws.com/155304_cc51f448116744069664b35e7762999f.html.

[3] Knime website. https://www.knime.com/.

[4] Leaflet package. https://cran.r-project.org/web/packages/leaflet/index.html.

[5] R studio. https://www.rstudio.com/.